

Chapter 4.1

Resources and Data Storage



Contents

Main vs Background Thread

Saving Data

- Data File

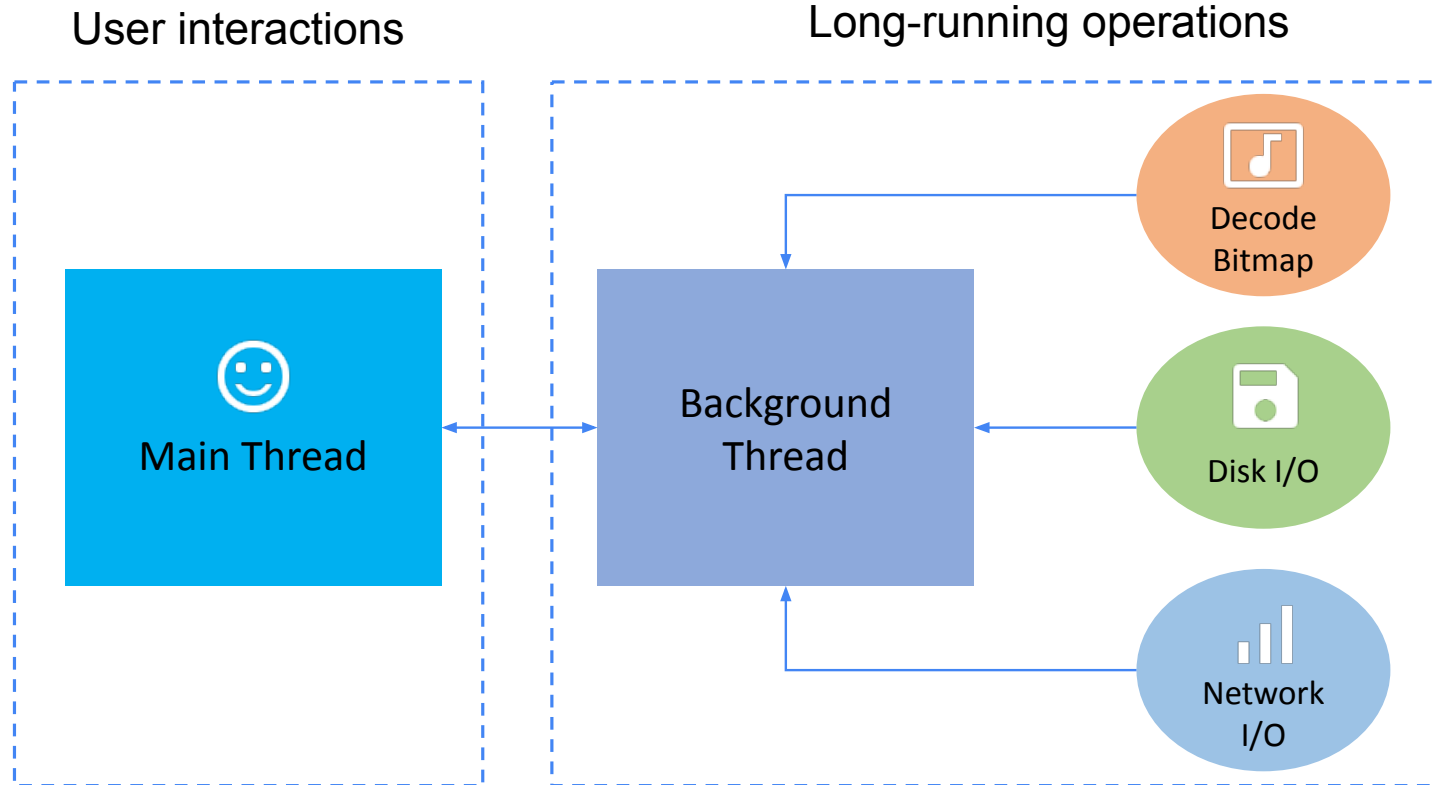
- Shared Preferences

- SQLite

Internal vs External Storage

AI Driven Mobile App Development : On-device vs Cloud Intelligence

Main vs Background Thread



Main Thread

Also called **UI Thread**, is responsible for **rendering the UI** and handling **user interactions**.

All UI updates and user input handling occur on this thread.

Long-running tasks, such as network requests or complex calculations, can **block the main thread**, leading to UI freezes and poor user experience.



Background Thread

Used to perform background tasks that **don't directly impact the UI**.

By offloading tasks to background threads, you ensure the main thread remains responsive.

Common Use Cases: Network requests, database operations, complex calculations, file I/O



Background Thread

Flutter uses the **Future** class to perform background thread.

File I/O operations, such as reading and writing files, are inherently asynchronous.

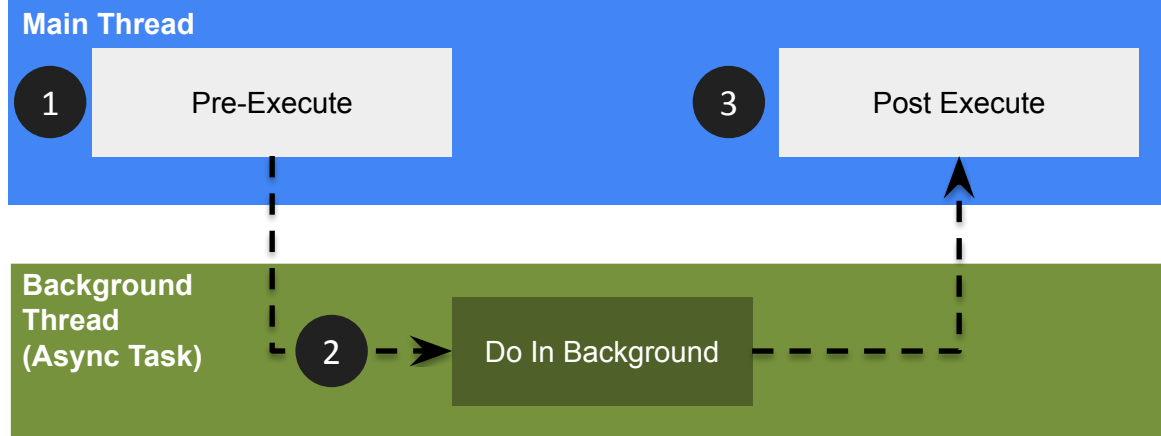
They can take varying amounts of time, and if not handled correctly, they can block the main thread, leading to a frozen UI.



Background Thread

It performs background operations and publish results on UI thread

Future is a class for working with async operations. A Future object represents a potential value or error that will be available at some time in the future.

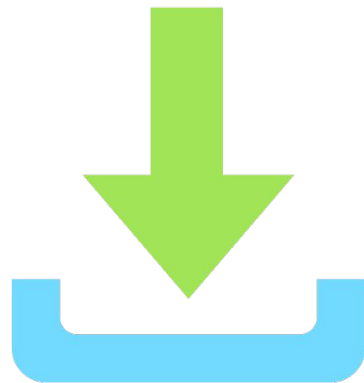


Asynchronous programming: futures, async, await

Asynchronous operations let your program **complete work while waiting for another operation to finish**.

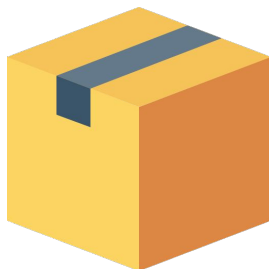
Here are some common asynchronous operations:

- Fetching data over a network.
- Writing to a database.
- Reading data from a file.

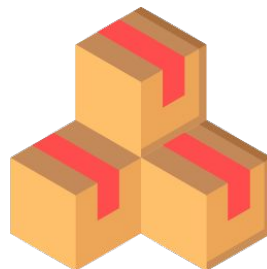


Asynchronous programming: futures, async, await

Asynchronous tasks produce result as a Future (single item) or Stream (multiple items).



Future



Stream

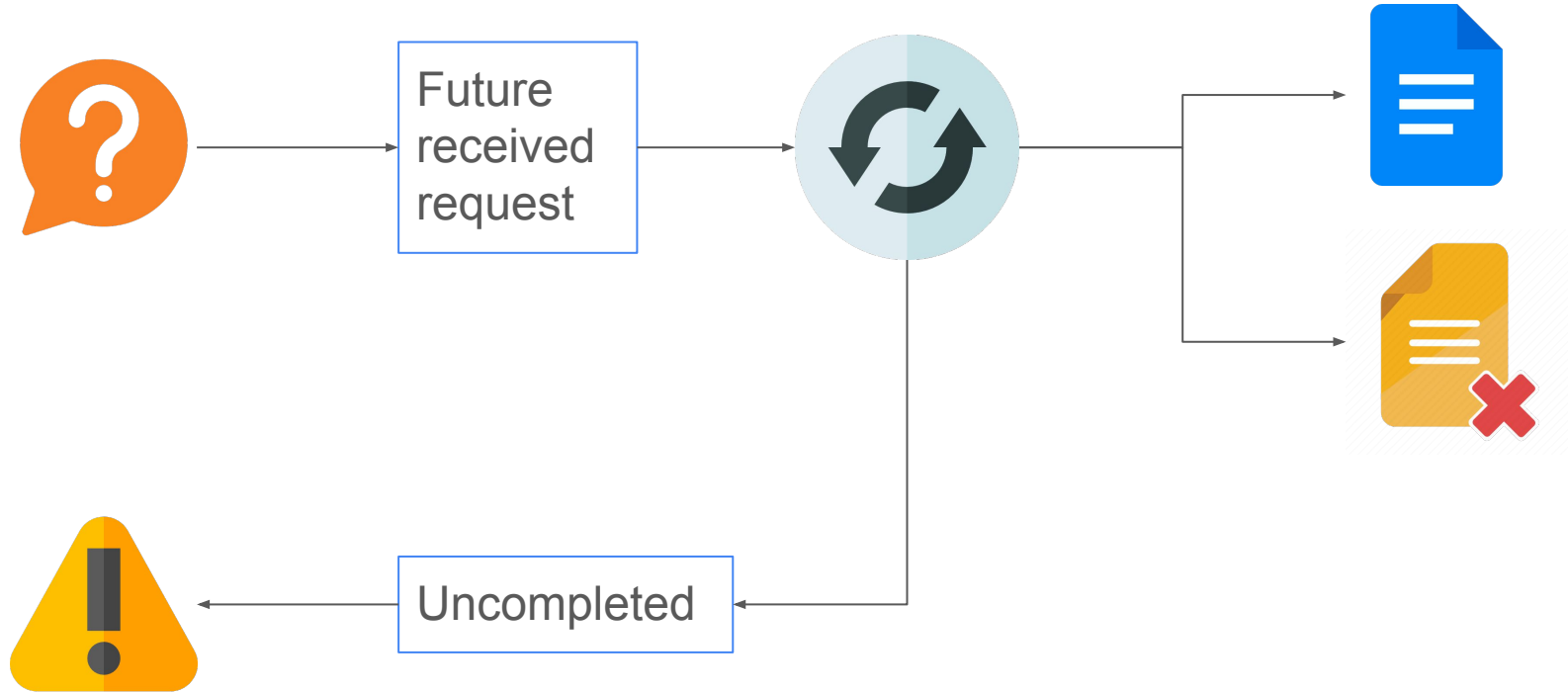
To interact with these asynchronous results, you can use the async and await keywords.

Future

A Future represents the **result** of an asynchronous operation, and can have two states:

Uncompleted	Completed
Returns an uncompleted future.	Complete with a value type <code>Future<T></code> . <code>T</code> could be any valid data type. E.g. <code>String</code>
That Future is waiting for the function's asynchronous operation to finish or to throw an error.	Complete with an error. The future type is <code>Future <void></code>

Future



Future

A Future represents the result of an asynchronous operation, and can have two states:

```
void main() {  
  fetchUserOrder();  
  print('Fetching user order...');  
}
```

Output:
Fetching user order...
Large Latte

```
Future<void> fetchUserOrder() {  
  return Future.delayed(const Duration(seconds: 2), () => print('Large Latte'));  
}
```

Async and Await

The async and await keywords provide a declarative way to define asynchronous functions and use their results.

Remember these two basic guidelines when using async and await:

1. Add async before the function body
2. Insert await keyword in async functions.

**Async before, that's the rule,
Await inside, keep it cool.**

**If you forget, your code's a
mess,
So async before, and await,
yes!"**

Async and Await

```
Future<void> main() async {  
    print('Fetching user order...');  
    print(await createOrderMessage());  
}
```

```
Future<String> createOrderMessage() async {  
    var order = await fetchUserOrder();  
    return 'Your order is: $order';  
}
```

```
Future<String> fetchUserOrder() =>  
    Future.delayed(  
        const Duration(seconds: 2), () => 'Large Latte',  
    );
```

Output:
Fetching user order...
Your order is: Large Latte

Async and Await

```
Future<String> createOrderMessage() async {  
  try{  
    var order = await fetchUserOrder();  
    return 'Your order is: $order';  
  }catch(e){  
    return 'Failed to obtain order';  
  }  
}
```

You can write try-catch clauses in synchronous code to handle errors in an async function.

Stream

Stream returns **multiple items**. You may use the await for iterates over the events of a stream like a for loop.

```
Future<int> sumStream(Stream<int> stream) async {  
    var sum = 0;  
  
    await for (final value in stream) {  
        sum += value;  
    }  
  
    return sum;  
}
```

Stream

In some cases, an error happens before the stream is done; perhaps the network failed while fetching a file from a remote server.

We may use try-catch to handle errors associated with a stream.

```
Future<int> sumStream(Stream<int> stream) async {  
    var sum = 0;  
  
    try {  
        await for (final value in stream) {  
            sum += value;  
        }  
    } catch(e){  
        return -1;  
    }  
    return sum;  
}
```

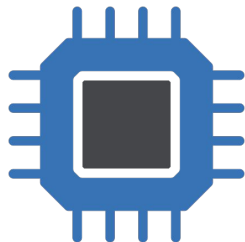
Class Code: 15 98 19



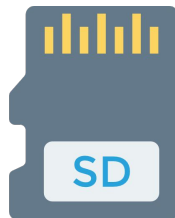
Saving Data

Saving Data

Flutter, being a cross-platform framework, provides mechanisms to access both **internal** and **external storage**.



Internal



External

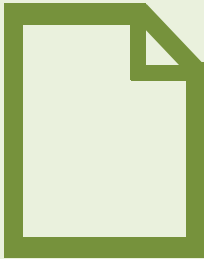
This allows you to store app **data locally**, ensuring persistence even when the app is offline.

Saving Data

Data Files

Private or Public

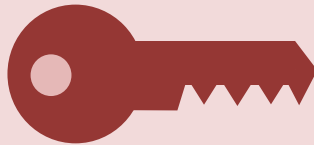
Data file; text, sound, images, and etc



Shared Preferences

Private or Public

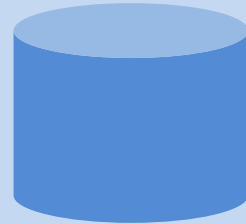
Key-value pair



SQLite

Private

Repeating and structured data



Data File

Data File

Larger amounts of data, structured data (like JSON), files (like PDF or image) downloaded from the internet, user-generated content.

Use cases:

- Persist data across app launches
- Download data from the internet and save it for later offline use

Data File

1. Find the correct local path.
2. Create a reference to the file location.
3. Write data to the file.
4. Read data from the file.

Data File

1: Find the correct local path:

Combine the path_provider and dart:io libraries.

```
import 'package:path_provider/path_provider.dart';  
// ...  
Future<String> get _localPath async {  
    final directory = await getApplicationDocumentsDirectory();  
  
    return directory.path;  
}
```

Data File

2. Create a reference to the file location:

```
Future<File> get _localFile async {  
    final path = await _localPath;  
    return File('$path/counter.txt');  
}
```

Data File

3. Check storage status:

```
Future<void> checkExternalStorage() async {  
    final directory = await getExternalStorageDirectory();  
  
    if (directory != null) {  
        print('External storage is available: ${directory.path}');  
    } else {  
        print('External storage is not available.');    }  
}
```

Data File

4. Write data to the file:

```
Future<File> get _localFile async {  
  final path = await _localPath;  
  return File('$path/counter.txt');  
}
```

Shared Preferences

Shared Preferences

Shared Preferences are suitable for storing small collection of key-values.

Limitations:

- Only primitive types can be used: `int`, `double`, `bool`, `String`, and `List<String>`.
- It's not designed to store large amounts of data.
- There is no guarantee that data will be persisted across app restarts.

Shared Preferences

1. Add the dependency:

Include the `shared_preferences` package in your *pubspec.yaml* file. Add this line under the dependencies section:

```
shared_preferences: ^2.0.13 # Adjust version if needed
```

2. Import the library:

In your Dart code, import the `shared_preferences` library to access its functionalities.

```
import 'package:shared_preferences/shared_preferences.dart';
```


Shared Preferences

3. Saving Data:

Shared Preferences can store various data types including:

Strings (`String`)

Integers (`int`)

Doubles (`double`)

Booleans (`bool`)

Shared Preferences

Use the appropriate method provided by SharedPreferences to save your data:

- `setString(key, value)`: Saves a string value.
- `setInt(key, value)`: Saves an integer value.
- `setDouble(key, value)`: Saves a double value.
- `setBool(key, value)`: Saves a boolean value.

Shared Preferences

```
Future<void> saveData(String name, int age) async {  
    final prefs = await SharedPreferences.getInstance();  
    await prefs.setString('name', name);  
    await prefs.setInt('age', age);  
    print('Data saved successfully!');  
}
```

Important: Since these methods return futures, it's recommended to use `async` and `await` keywords to ensure data is saved before proceeding.

Shared Preferences

4. Retrieving Data:

To retrieve data, use the corresponding getter methods based on the data type you saved:

- `getString(key)`: Retrieves a string value.
- `getInt(key)`: Retrieves an integer value.
- `getDouble(key)`: Retrieves a double value.
- `getBool(key)`: Retrieves a boolean value.

These methods return the value or null if the key doesn't exist. You can also provide a default value in case the key is missing.

Shared Preferences

```
Future<String> getName() async {  
    final prefs = await SharedPreferences.getInstance();  
  
    // Default if not found  
    final name = prefs.getString('name') ?? 'Default Name';  
    return name;  
}
```

Shared Preferences

5. Removing Data:

Use the `remove(key)` method to remove a specific key-value pair from Shared Preferences.

```
Future<void> removeData(String key) async {  
    final prefs = await SharedPreferences.getInstance();  
    await prefs.remove(key);  
    print('Data with key $key removed');  
}
```

SQLite

SQLite

SQLite is a lightweight, embedded SQL database engine that's perfect for mobile applications.



SQLite

It provides faster inserts, updates, and queries for large amounts of data on the local storage.

Flutter offers a plugin called `sqflite` to interact with SQLite databases.

The `sqflite` package only supports apps that run on macOS, iOS and Android.

SQLite

Step 1: Add the dependency to your `pubspec.yaml` file:

dependencies:

sqflite: ^2.3.3+1

path: ^1.9.0

Or run

```
$flutter pub add sqflite path
```

SQLite

Step 2: Import packages:

```
import 'dart:async';
```

```
import 'package:sqflite/sqflite.dart';
```

SQLite

Step 3: Define a data model:

```
class Note {  
    final int id;  
    final String title;  
    final String description;  
  
    const Note ({  
        required this.id,  
        required this.title,  
        required this.description,  
    });  
}
```

```
// Convert a Note into a Map.  
Map<String, Object?> toMap() {  
    return {  
        'id': id,  
        'title': title,  
        'description': description,  
    };  
}  
  
// Implement toString  
@override  
String toString() {  
    return 'Note{id: $id, title: $title,  
        description: $description}';  
}
```

SQLite

Step 4: Open database:

```
// Avoid errors caused by flutter upgrade.  
WidgetsFlutterBinding.ensureInitialized();  
  
// Open the database and store the reference.  
final database = openDatabase(  
  // Set the path to the database.  
  join(await getDatabasesPath(), 'note.db'),  
);
```

SQLite

Step 5: Create table:

```
final database = openDatabase(  
    // Set the path to the database.  
    join(await getDatabasesPath(), 'note.db'),  
  
    // When the database is first created, create a table to store notes.  
    onCreate: (db, version) {  
        // Run the CREATE TABLE statement on the database.  
        return db.execute(  
            'CREATE TABLE notes(id INTEGER PRIMARY KEY, title TEXT, description TEXT)',  
        );  
    },  
    // Set the version.  
    version: 1,  
);
```

SQLite

Step 6: Insert

```
Future<void> insertNote(Note note) async {  
    // Get a reference to the database.  
    final db = await database;  
  
    // Insert the Note into the correct table.  
    await db.insert(  
        'notes',  
        note.toMap(),  
        conflictAlgorithm: ConflictAlgorithm.replace,  
    );  
}
```

```
var note1 = Note(  
    id: 0,  
    title: 'Note 1',  
    description: 'Desc 1',  
);  
  
await insertNote(note1);
```

SQLite

Step 6: Retrieve notes

```
Future<List<Note>> notes() async {  
    final db = await database; // Get a reference to the database.  
  
    // Query the table for all the dogs.  
    final List<Map<String, Object?>> noteMaps = await db.query('notes');  
  
    // Convert each note's fields into a list of Note  
    return [  
        for (final {'id': id as int, 'title': title as String,  
                    'description': description as String,  
                    } in noteMaps)  
            Note(id: id, title: title, description: description),  
    ];  
}  
...  
print(await notes());
```


SQLite

Step 6: Retrieve notes

```
Future<List<Note>> notes() async {  
    final db = await database; // Get a reference to the database.  
  
    // Query the table for all the dogs.  
    final List<Map<String, Object?>> noteMaps = await db.query('notes');  
  
    // Convert each note's fields into a list of Note  
    return [  
        for (final {'id': id as int, 'title': title as String,  
                    'description': description as String,  
                    } in noteMaps)  
            Note(id: id, title: title, description: description),  
    ];  
}  
...  
print(await notes());
```

SQLite

Step 7: Update notes

```
Future<void> updateNote(Note note) async {  
    // Get a reference to the database.  
    final db = await database;  
  
    // Update the given Note.  
    await db.update(  
        'notes',  
        note.toMap(),  
        // Ensure that the Note has a matching id.  
        where: 'id = ?',  
        // Pass the Note's id as a whereArg to prevent SQL injection.  
        whereArgs: [note.id],  
    );  
}  
...  
await updateNote(newNote);
```

SQLite

Step 8: Delete notes

```
Future<void> deleteNote(int id) async {  
    // Get a reference to the database.  
    final db = await database;  
  
    // Remove the Note from the database.  
    await db.delete(  
        'notes',  
        // Use a `where` clause to delete a specific note.  
        where: 'id = ?',  
        // Pass the Note's id as a whereArg to prevent SQL injection.  
        whereArgs: [id],  
    );  
}
```

...

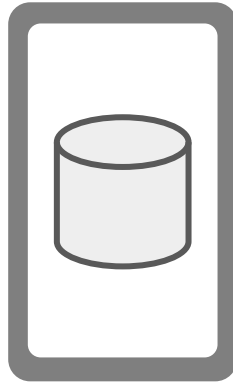
```
await deleteNote(newNote.id);
```

Internal vs External Storage

Internal Storage

Storage that is directly accessible to your app.

Typically used for storing app data, preferences, and cache.



Internal Storage

An app's internal storage directory is specified by the app's package name

No permission is required to perform read/write operations

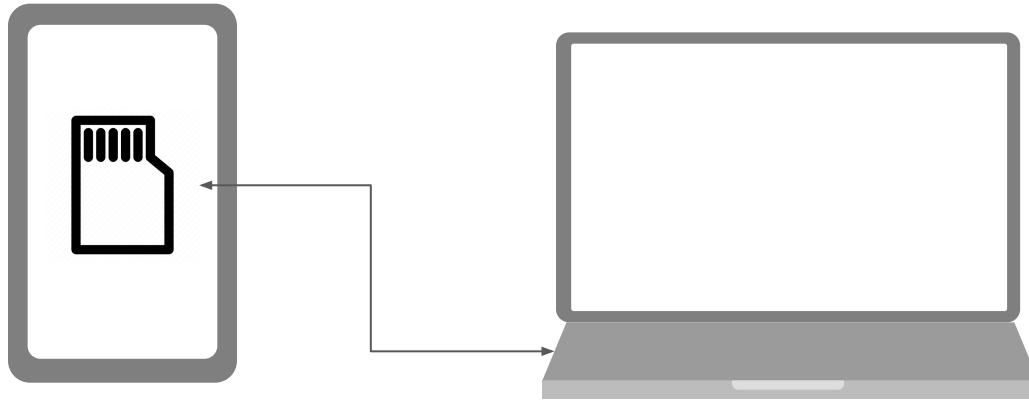
Suitable for:

- Shared Preferences
- SQLite Database

External Storage

Storage that is accessible to multiple apps on the device and outside of the device.

Typically used for storing large files or media.



External Storage

Feature	Android	iOS
Expandable Storage	Yes	No (generally)
Removable	Possible (SD Card)	No
File System Access	More flexible	More restricted
Permissions	Requires explicit user permission	More controlled by the system
Data Sharing	Easier to share files between apps	More limited sharing options

External Storage

1. **Permissions:** Ensure that your app has the necessary permissions to access storage.
2. **Data Security:** Consider encrypting sensitive data before storing it.
3. **Performance:** Optimize file I/O operations to avoid performance bottlenecks.
4. **User Experience:** Provide clear feedback to the user during file operations, such as progress indicators.

AI Driven Mobile App Development

AI Driven Mobile App Development

Since 2025, mobile app development has moved beyond the "mobile-first" mindset into the AI-Native era.

We no longer just build apps that users use; we build apps that understand, predict, and act on behalf of the user.

Two implementation models:

1. On-Device APIs (zero-cost, hyper-private, offline) and
2. Cloud-Based APIs (high-intelligence, massive context windows)

AI Driven Mobile App Development

Feature	On-Device APIs (AICore / Foundation Models)	Cloud APIs (Gemini API / OpenAI)
Cost	Free (Runs on user's hardware)	Metered (Pay per token/request)
Latency	Extremely low (Near-instant)	Dependent on network connection
Privacy	Maximum (Data never leaves device)	Dependent on data processing agreements
Capabilities	Core tasks (Summarize, rewrite, simple vision)	Advanced tasks (Deep reasoning, massive scale)
Offline Support	Yes	No

AI Driven Mobile App Development

With the power of modern chips, apps are processing data locally instead of in the Cloud to ensure:

1. Privacy: Sensitive data (health, personal messages, financial info) never leaves the device.
2. Latency: Features like real-time translation and image editing happen instantly without waiting for a server.
3. Offline Capability: AI features remain functional even in "dead zones" or during travel.

AI Driven Mobile App Development

Developers are increasingly using frameworks like Core ML (Apple) and TensorFlow Lite (Google) to bundle lightweight models directly into their binaries.



TensorFlowLite

Find out more

Data File, <https://docs.flutter.dev/cookbook/persistence/reading-writing-files>

Key-Value pair, <https://docs.flutter.dev/cookbook/persistence/key-value>

SQLite, <https://docs.flutter.dev/cookbook/persistence/sqlite>